Discrete Mathematics

Jaehaerys Wang

2026 年 3 月 14 日

目录

1 Chapter 1: Proofs **3**
1.1 什么是证明？ (What is a Proof?) 3
1.2 良序原理 (The Well Ordering Principle) 4
1.3 逻辑公式 (Logical Formulas) 4
1.4 数学数据类型 (Mathematical Data Types) 5
1.5 归纳法 (Induction) 6
1.6 递归数据类型 (Recursive Data Types) 7
1.7 无限集合 (Infinite Sets) 7

2 Chapter 2: Structures **8**
2.1 数论与密码学 (Number Theory) 8
2.2 有向图与偏序关系 (Directed Graphs & Partial Orders) 11
2.3 通信网络 (Communication Networks) 13
2.4 简单图 (Simple Graphs) 13
2.5 平面图 (Planar Graphs) 15

3 Chapter 3: Counting **16**
3.1 求和与渐近 (Sums and Asymptotics) 16
3.2 基数规则 (Cardinality Rules) 18
3.3 生成函数 (Generating Functions) 19

1 Chapter 1: Proofs

证明

本书的第一部分“证明”(Proofs)旨在为计算机科学提供坚实的数学和逻辑基础。该部分通过七个章节, 系统地介绍了如何运用数学模型和方法来分析问题、建立确定性, 并确保软件或算法的正确性。

从最基本的逻辑推演和证明技巧起步, 逐步深入到良序原理和多种归纳法(尤其是与算法分析息息相关的状态机和结构归纳法), 最后上升到探讨无穷集合与计算边界的理论极限。这一部分为后续关于图论、图结构、计数理论和概率论等计算机科学内容的学习打下了严密的数学基石。

1. 什么是证明？ (What is a Proof?)
2. 良序原理 (The Well Ordering Principle)
3. 逻辑公式 (Logical Formulas)
4. 数学数据类型 (Mathematical Data Types)
5. 归纳法 (Induction)
6. 递归数据类型 (Recursive Data Types)
7. 无限集合 (Infinite Sets)

1.1 什么是证明？ (What is a Proof?)

基础概念: 介绍了**命题(Proposition)**, 即非真即假的陈述, 和**谓词(Predicate)**, 即其真值取决于变量的命题。

公理化方法与推导: 证明是从一组**公理(Axioms)**出发, 通过逻辑推导得出的结论链条。

常用证明模式: 详细讲解了几种基础的证明方法, 包括:

- **蕴涵证明(Proving an Implication)**:证明“如果P, 则Q”及其逆否命题。
- **当且仅当证明(If and Only If)**:证明双向蕴涵关系。
- **分类讨论(Proof by Cases)**:将复杂问题拆分为不同的情况分别证明。
- **反证法(Proof by Contradiction)**:假设命题为假并推导出逻辑矛盾。

经典的“伪证明”(Bogus Proofs)与逻辑陷阱: 强调培养**识别错误证明**的能力。

- **图形伪证明**:隐藏假设或误用图形来误导视觉判断。
- **计算伪证明**:忽视多义性或不当简化计算步骤。
- **归纳法伪证明**:骨牌断裂或错误的归纳假设。

1.2 良序原理 (The Well Ordering Principle)

核心定义: 任何非空的非负整数集合($\mathbb{N}$)都必然包含一个最小元素。虽然听起来极其直观, 但它是离散数学中极其强大的证明工具。

应用: 提供了一种标准模板, 通过假设存在反例集合, 并由良序原理得出“最小反例”, 随后推导出矛盾来证明原命题。例如, 用于证明算术基本定理(整数可以分解为素数乘积)。

Well Ordering Principle Proofs: To prove $\forall n \in \mathbb{N}, P(n)$ using WOP:

1. Assume for contradiction that the set $S = \{n \in \mathbb{N} : \neg P(n)\}$ is non-empty.
2. By WOP, S has a least element, say m.
3. Show that $P(k)$ holds for all $k < m$ (this is often done by using the definition of m as the smallest counterexample).
4. Use the fact that $P(k)$ holds for all $k < m$ to derive a contradiction, thus proving that S must be empty and therefore $\forall n \in \mathbb{N}, P(n)$ holds.

1.3 逻辑公式 (Logical Formulas)

命题逻辑: 探讨了如何使用 AND($\wedge$)、OR($\vee$)、NOT($\neg$)、XOR($\oplus$)、IMPLIES($\rightarrow$)、IFF($\leftrightarrow$) 等逻辑连接词来组合命题, 并通过真值表(Truth Table)来判断公式的等价性和有效性。

DeMorgan 定律: 介绍了 DeMorgan 定律, 揭示了 NOT 与 AND/OR 之间的关系, 即 $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ 和 $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$。

可满足性问题(SAT Problem): 提及了判断命题公式是否可满足的计算复杂性问题。

- **可满足性(Satisfiability)**: 一个命题公式P是可满足的(有解的), 当且仅当在某些变量赋值下它为真, 若P是可满足的则$\neg P$也是可满足的。
- **有效性(Validity)**: 一个命题公式P是有效的(永远为真), 当且仅当在所有变量赋值下它为真, 若P是有效的则$\neg P$是不可满足的。
- **逻辑等价(Logical Equivalence)**: 两个命题公式P和Q是逻辑等价的, 当且仅当在所有变量赋值下它们具有相同的真值。
- **蕴涵(Implication)**: 一个命题公式P蕴涵另一个命题公式Q, 当且仅当在所有变量赋值下, 如果P为真则Q也为真。
- **NP-完全(NP-Complete)**: NP指的是非确定性多项式时间复杂度类, 可满足性问题是第一个被证明为 NP-完全的问题, 这意味着它是所有 NP 问题中最难的一个, 如果能找到一个多项式时间算法来解决 SAT 问题, 那么所有 NP 问题都可以在多项式时间内解决。
- **$P = NP$ 问题**: 千禧年大奖难题之一, 涉及是否所有可验证的解都可以在多项式时间内找到。解释了为什么快速检查命题公式是否可满足不仅是逻辑问题, 更是理论计算机科学中最核心的未解之谜。

谓词公式: 引入了**全称量词**($\forall$, 对于所有)和**存在量词**($\exists$, 存在), 解释了多重量词嵌套时的顺序问题及其对应的逻辑含义。

关于逻辑公式的元定理(Meta-theorems):

- **完备性定理(Completeness Theorem)**: 任何在语义上有效的逻辑公式都可以通过形式证明系统来证明。换句话说, 如果一个命题在所有模型中都为真, 那么存在一个证明该命题的形式证明。即好消息是所有有效的逻辑公式都可以被证明。
- **不可判定性定理(Undecidability Theorem)**: 不存在一个算法能够判断所有逻辑公式是否有效。换句话说, 无法构造一个程序来决定任意给定的命题是否在所有模型中为真。即坏消息是不存在一个算法能判断所有逻辑公式是否有效。
- **哥德尔不完全性定理(Gödel's Incompleteness Theorems)**: 在任何足够强大的形式系统中, 存在无法被证明或反驳的命题。这表明了形式系统的局限性, 即使是数学也无法完全自洽地证明所有真命题。

1.4 数学数据类型 (Mathematical Data Types)

基础结构: 严格定义了计算机科学中不可或缺的数学结构:

- **集合(Sets)**: 集合是元素的无序集合, 强调了集合的基本操作(如并集$\cup$、交集$\cap$、差集$\setminus$、补集$\overline{A}$)和关系(如子集$\subseteq$、超集$\supseteq$), 以及特殊集合(如空集$\emptyset$)。
- **序列(Sequences)**: 序列是元素的有序列表, 强调了序列的定义、长度以及常见操作(如连接、切片)。
- **函数(Functions)**: 函数是从一个集合到另一个集合的映射, 强调了函数的定义域、值域、复合函数等概念, **映射(Mapping)**是函数的一种特殊类型。
- **二元关系(Binary Relations)**: 二元关系是集合元素之间的关系, 强调了关系的性质(如自反性、对称性、传递性)以及特殊类型的关系(如等价关系、偏序关系)。

映射法则与基数:

- **基数(Cardinality)**: 集合的基数($|A|$)是集合中元素的数量。对于有限集合, 基数就是元素的个数; 对于无限集合, 基数用特殊的符号表示, 如自然数集$\mathbb{N}$的基数记为$\aleph_0$(阿列夫零), 实数集$\mathbb{R}$的基数大于$\aleph_0$, 通常记作$c = 2^{\aleph_0}$。
- **有限集合(Finite Sets)**: 有限集合是指其基数为非负整数的集合, 即包含有限数量元素的集合。
- **无限集合(Infinite Sets)**: 无限集合是指包含无限数量元素的集合。
- **可数集合(Countable Sets)**: 可数集合是指其基数与自然数集合$\mathbb{N}$相同的集合, 即存在双射从该集合到$\mathbb{N}$的集合。
- **不可数集合(Uncountable Sets)**: 不可数集合是指其基数大于自然数集合$\mathbb{N}$的集合, 即不存在双射从该集合到$\mathbb{N}$的集合。
- **满射(Surjection)**: 存在一个函数$f : A \to B$使得对于每个$b \in B$, 至少存在一个$a \in A$使得$f(a) = b$。如果存在满射从集合A到集合B, 则称集合A的基数大于或等于集合B的基数。

- **单射(Injection)**: 存在一个函数$f: A \to B$使得对于每个$a_1, a_2 \in A$, 如果$f(a_1) = f(a_2)$, 则$a_1 = a_2$。如果存在单射从集合A到集合B, 则称集合A的基数小于或等于集合B的基数。
- **双射(Bijection)**: 存在一个函数$f: A \to B$既是单射又是满射, 即对于每个$b \in B$, 存在唯一的$a \in A$使得$f(a) = b$. 如果存在双射从集合A到集合B, 则称集合A和集合B具有相同的基数。
- **基数比较原则(Cardinality Comparison Principle)**: 如果存在单射从集合A到集合B, 则$|A| \leq |B|$；如果存在满射从集合A到集合B, 则$|A| \geq |B|$；如果存在双射从集合A到集合B, 则$|A| = |B|$。
- **施罗德-伯恩斯坦定理(Schröder-Bernstein Theorem)**: 如果存在单射从集合A到集合B, 同时存在单射从集合B到集合A, 则存在双射从集合A到集合B, 因此集合A和集合B具有相同的基数。

1.5 归纳法 (Induction)

普通归纳法(Ordinary Induction): 归纳法是证明某属性$P(n)$对所有非负整数n成立的核心方法。

- **基础情况(Base Case)**: 验证$P(0)$或$P(1)$等初始值是否成立。
- **归纳步骤(Inductive Step)**: 假设$P(k)$对某个任意的非负整数k成立, 证明$P(k+1)$也成立。则根据归纳法原理, $P(n)$对所有非负整数n成立。

强归纳法(Strong Induction): 强归纳法允许在归纳步骤中假设此前所有步骤均成立, 从而简化证明。即在证明$P(k+1)$时, 可以使用$P(0), P(1), \ldots, P(k)$的假设, 而不仅仅是$P(k)$。

三大证明方法的等价性: **良序原理(WOP)、普通归纳法和强归纳法在逻辑上是完全等价的。** 任何一个用强归纳法写的证明, 都可以通过添加全称量词机械地转换为普通归纳法证明；反之亦然。

状态机(State Machines): 状态机是一种step-by-step过程的抽象模型, 适用于描述算法的执行过程。通过定义状态转移规则, 可以使用归纳法来证明算法在每一步都满足某些不变性条件, 从而确保算法的正确性和终止性。

- **Floyd不变性原理(Floyd's Invariant Principle)**: 在状态机的每一步中, 如果某个属性(如数值)保持不变或满足特定条件, 那么可以通过归纳法证明该属性在整个过程中始终成立。
- **衍生变量(Derived Variables)**: 定义一个严格递减的变量, 如果该变量只能取非负整数值, 那么根据良序原理, 状态机必然会在有限步骤内终止。

1.6 递归数据类型 (Recursive Data Types)

递归定义(Recursive Definitions): 递归数据类型是通过自身定义的结构, 常见于计算机科学中的字符串、树、表达式等。证明递归数据类型的性质通常使用结构归纳法。

结构归纳法(Structural Induction): 结构归纳法是归纳法的一种推广, 适用于递归定义的数据结构。证明时只需涵盖基础元素, 并证明构造操作能保持该属性不变即可。

- **基础情况(Base Case)**: 验证递归定义中的基本元素(如空字符串、单个节点)是否满足属性。
- **构造步骤(Constructive Step)**: 假设属性对某些元素成立, 证明通过递归构造操作生成的新元素也满足该属性。

1.7 无限集合 (Infinite Sets)

对角线论证(Diagonal Argument): 康托尔对角线法, 通过构造一个与任意给定列表中的元素都不同的元素, 证明了实数集的不可数性。这个方法不仅揭示了无限集合的层次结构, 还在计算机科学中被用来证明停机问题的不可判定性, 以及罗素悖论等逻辑悖论的存在。

康托尔定理(Cantor's Theorem): 康托尔定理表明, 对于任何集合A, 其幂集$\mathcal{P}(A)$的基数严格大于A的基数, 即$|\mathcal{P}(A)| > |A|$。这一结果揭示了无限集合的层次结构, 说明了存在不同大小的无限集合, 并且没有一个最大无限集合。这一发现对数学基础和集合论的发展产生了深远影响。

停机问题(Halting Problem): 通过对角线论证, 证明了不存在一个算法能够判断任意程序是否会在有限时间内停止。这一结果表明了计算的根本限制, 说明了某些问题是无法通过算法解决的, 进而说明**完美的类型检查器或代码优化器**在在逻辑上是不存在的。

罗素悖论(Russell's Paradox): 罗素悖论揭示了在集合论中, 如果允许任意集合的存在, 就会导致矛盾。例如, 考虑集合R, 定义为所有不包含自身的集合的集合。问题是:R是否包含自身? 如果R包含自身, 那么根据定义它不应该包含自身; 如果R不包含自身, 那么根据定义它应该包含自身。这种矛盾表明了需要对集合的定义进行限制, 最终促使数学家发展出更严格的集合论公理系统, 如 ZFC 集合论, 以避免类似的悖论。

ZFC 集合论: 通过一系列严格的公理来定义集合的存在和操作, 避免了罗素悖论等逻辑矛盾, 使得集合论成为一个坚实的数学基础。

- **Zermelo-Fraenkel 公理(ZF Axioms)**: 包括空集公理、配对公理、并集公理、幂集公理、替代公理等, 定义了集合的基本性质和构造方法。
- **选择公理(Axiom of Choice)**: 断言对于任何非空集合的集合, 都存在一个选择函数能够从每个集合中选择一个元素。选择公理在许多数学领域中具有重要应用, 但也引发了一些争议, 因为它允许构造一些非构造性的对象。

2 Chapter 2: Structures

结构

本书的第二部分“结构”(Structures)是离散数学与计算机科学实际应用结合最紧密的部分。它研究了由不同规则和组件构成的离散数学对象(如数字系统、各类图结构、通信网络等)。
通过对这些结构的深入分析, 不仅揭示了它们的内在规律和性质, 还展示了它们在算法设计、数据组织、网络通信等计算机科学核心领域中的广泛应用。

1. 数论与密码学 (Number Theory)
2. 有向图与偏序关系 (Directed Graphs & Partial Orders)
3. 通信网络 (Communication Networks)
4. 简单图 (Simple Graphs)
5. 平面图 (Planar Graphs)

2.1 数论与密码学 (Number Theory)

整除性(Divisibility): 是数论的核心概念, 定义了一个整数a是否能被另一个整数b整除(即存在整数k使得$a = kb$)。

- $a|b$表示a整除b, 即存在整数k使得$b = ak$。
- 整除关系具有传递性: 如果$a|b$且$b|c$, 则$a|c$。
- 整除关系具有反对称性: 如果$a|b$且$b|a$, 则$a = \pm b$。
- 整除关系具有自反性: 对于任何整数a, $a|a$。
- 整除关系具有线性性: 如果$a|b$且$a|c$, 则对于任何整数x和y, $a|(xb + yc)$。
- 整除关系具有分配性: 如果$a|b$且c是任意整数, 则$a|(bc)$。
- 整除性是理解数的结构和性质的基础, 也是许多算法(如欧几里得算法)的核心。

最大公约数(GCD):

- 公共因子(common divisor)是两个或多个整数的共同因子, 即能同时整除这些整数的数。
- 最大公约数(The Greatest Common Divisor)是所有公共因子中最大的一个, 即能同时整除这些整数的最大整数。GCD在数论中具有重要意义, 例如在分数约简、求解线性方程、以及现代密码学中的密钥生成等方面都有广泛应用。
- $\gcd(a, b)$表示a和b的最大公约数。
- 最大公约数具有以下性质:
  1. $\gcd(a, 0) = |a|$对于任何整数a。
  2. $\gcd(a, b) = \gcd(b, a)$ (对称性)。
  3. $\gcd(a, b) = \gcd(|a|, |b|)$ (绝对值不影响GCD)。

4. $\gcd(a,b) = \gcd(a-b,b)$ (减法性质)。
5. $\gcd(a,b) = \gcd(a \mod b, b)$ (模运算性质)。
6. $\gcd(a,b) \cdot \mathrm{lcm}(a,b) = |a \cdot b|$ (GCD与LCM的关系)。
7. 如果$\gcd(a,b) = 1$, 则称a和b互质(coprime)。

欧几里得算法(Euclidean Algorithm): 一种高效计算两个整数的GCD的方法。

- 基于以下原理: $\gcd(a,b) = \gcd(b, a \mod b)$。
- 通过不断地将较大的数替换为较小的数的余数, 最终可以得到最大公约数。
- 时间复杂度为$O(\log \min(a,b))$, 是计算机科学中最经典的算法之一。

粉碎机算法(The Pulverizer): 是欧几里得算法的扩展。

- 通过回溯欧几里得算法的步骤, 可以找到满足线性方程$ax + by = \gcd(a,b)$的整数解(x,y)。
- 该算法不仅计算GCD, 还提供了GCD的线性组合表示, 这在数论和密码学中具有重要应用。
- 在 RSA 加密算法中, 粉碎机算法用于计算私钥, 通过求解线性方程来找到满足特定条件的整数解。

唯一分解定理(Unique Factorization Theorem): 每个大于1的整数都可以唯一分解为质数的乘积, 这为数论提供了坚实的基础, 也是许多算法(如RSA加密)的核心原理。

全等关系(Congruence): 在模算术中, 两个整数a和b如果在模m下具有相同的余数, 则称它们全等, 记作$a \equiv b \mod m$, 当且仅当m整除$a-b$。全等关系具有以下性质:

1. $a \equiv a \mod m$ (自反性)。
2. 如果$a \equiv b \mod m$, 则$b \equiv a \mod m$ (对称性)。
3. 如果$a \equiv b \mod m$且$b \equiv c \mod m$, 则$a \equiv c \mod m$ (传递性)。
4. 如果$a \equiv b \mod m$且c是一个整数, 则$(a+c) \equiv (b+c) \mod m$ (加法兼容性)。
5. 如果$a \equiv b \mod m$且c是一个整数, 则$ca \equiv cb \mod m$ (乘法兼容性)。
6. 如果$a \equiv b \mod m$且d是m的一个因子, 则$a \equiv b \mod d$ (模数的约简)。
7. 如果$ak \equiv bk \mod m$且$\gcd(k,m) = 1$, 则$a \equiv b \mod m$ (乘法约简)。
8. 如果$\gcd(a,m) = 1$, 则存在整数x使得$ax \equiv 1 \mod m$ (存在乘法逆元)。
9. **费马小定理(Fermat's Little Theorem)**: 如果p是一个质数且a是一个整数不被p整除, 则$a^{p-1} \equiv 1 \mod p$。这个定理在数论和密码学中具有重要应用。

欧拉函数(Euler's Totient Function): $\phi(n)$表示小于n且与n互质的正整数的数量, 即$\phi(n) = |\{k \in \mathbb{N} | 1 \le k < n, \gcd(k,n) = 1\}|$。欧拉函数在数论和密码学中具有重要应用, 特别是在 RSA 加密算法中。

- $\phi(n) = n \prod_{p|n}(1 - \frac{1}{p})$, 其中$p$是$n$的质因子。
- $\phi(ab) = \phi(a) \cdot \phi(b)$如果$\gcd(a,b) = 1$。
- $\phi(p) = p - 1$如果p是质数。
- $k^{\phi}(n) \equiv 1 \mod n$如果$\gcd(k,n) = 1$ (欧拉定理)。
- $\phi(p^k) = p^k - p^{k-1}$如果$p$是质数且$k \ge 1$。

模算术(Modular Arithmetic): 模算术是对整数进行除法后余数的运算系统, 其中整数被划分为等价类。

- 在模m下, 任何整数a都可以表示为一个余数r (0 ⩽ r ¡ m), 即$a \equiv r \mod m$。
- 模算术中的加法、减法和乘法运算都遵循全等关系的兼容性。
- **整数环(Z_n)**: 模算术可以形式化为整数环Z_n, 其中元素是从0到$n-1$的整数, 运算是模n的加法和乘法。
- 在整数环Z_n中, 只有当$\gcd(a,n)=1$时, 元素a才具有乘法逆元。
- **图灵密码(Turing's Code)**: 阿兰·图灵早期基于素数乘法的密码系统, 原理是将消息编码为整数, 并通过乘以一个大质数来加密。虽然这个系统在当时被认为是安全的, 但后来被证明存在安全漏洞, 特别是在模算术除法下的攻击方法, 使得该系统不再适用于现代密码学需求。

RSA 加密算法(RSA Encryption): RSA 是一种基于数论的公钥加密算法, 其安全性依赖于大数分解问题的困难性。

- 密钥生成: 选择两个大质数p和q, 计算$n = pq$和$\phi(n) = (p-1)(q-1)$, 选择一个公钥指数e使得$1 < e < \phi(n)$且$\gcd(e,\phi(n)) = 1$, 计算私钥指数d使得$ed \equiv 1 \mod \phi(n)$。
- 加密: 将消息m转换为整数形式, 计算密文$c = m^e \mod n$。
- 解密: 使用私钥指数d计算原始消息$m = c^d \mod n$。
- 数学原理: 对于任何消息m使得$\gcd(m,n)=1$, 都有$m^{ed} \equiv m \mod n$ (欧拉定理)。
- RSA 的安全性基于大数分解的困难性, 即没有已知的多项式时间算法能够有效地分解大整数n。

SAT 规约(SAT Reduction): 将大数分解问题规约为命题逻辑的可满足性问题(SAT 问题)。

- 通过构造一个命题公式, 其中变量表示可能的因子组合, 并使用逻辑连接词来表达分解的约束条件。
- 如果该命题公式是可满足的, 则存在一个满足条件的因子组合, 从而成功地分解了大整数。
- SAT 规约展示了数论问题与逻辑问题之间的深刻联系, 以及计算复杂性理论中不同问题之间的相互关系。不仅在理论计算机科学中具有重要意义, 还在实际应用中被用来解决各种组合优化问题, 包括密码分析、调度问题等。

2.2 有向图与偏序关系 (Directed Graphs & Partial Orders)

有向图(Directed Graphs): 由一组顶点和一组有向边组成的图, 可以表示事物间的二元关系, 记为$G = (V, E)$, 其中V是顶点集合, E是有向边集合, $E \subseteq V \times V$。

- **Vertex Degrees**: 在有向图中, 每个顶点有一个入度(in-degree)和一个出度(out-degree), 分别表示指向该顶点的边的数量和从该顶点出发的边的数量。

$$\begin{aligned} \text{indeg}(v) &= |\{u \in V | (u, v) \in E\}| \\ \text{outdeg}(v) &= |\{w \in V | (v, w) \in E\}| \\ \sum_{v \in V} \text{indeg}(v) &= \sum_{v \in V} \text{outdeg}(v) = |E| \end{aligned}$$

- **Walk**: 在有向图中, 从一个顶点到另一个顶点的路径称为walk, 其中允许重复顶点和边。
- **Path**: 没有重复顶点的walk称为path。
- **环(Cycle)**: 起点和终点相同的path称为cycle。
- **邻接表(Adjacency List)**: 用一个列表来表示每个顶点的邻居, 适用于稀疏图。
- **邻接矩阵(Adjacency Matrix)**: 用一个$|V| \times |V|$的矩阵来表示有向图. 如果存在边(u, v), 则矩阵中的对应位置为1, 否则为0。

$$(A_G)_{uv} = \begin{cases} 1 & \text{if } (u, v) \in E(G) \\ 0 & \text{otherwise} \end{cases}$$

- 两个顶点u和v之间最短的walk是一个path。
- 对于两个有向图G和H, 邻接矩阵$A_{G \circ H} = A_G \odot A_H$。所以计算$A_{G^N}$可以用矩阵快速幂来实现。
- $uG^N v$表示存在一个从u到v长度为N的walk。
- $uG^* v$表示存在一个从u到v的walk, 即u和v联通。$G^* = (G \cup G^0)^{N-1}$表示所有顶点对之间的联通关系, 其中G^0是自环, 可以在$\log N$时间内计算G^*。

有向无环图(Directed Acyclic Graphs): DAG 是没有任何正长度环的有向图, 常用于表示任务之间的依赖关系。

- **覆盖边(Covering Edges)**: 在 DAG 中, 如果存在一条边(u, v), 但不存在一个顶点w使得(u, w)和(w, v)都是边, 则称(u, v)是一个覆盖边。覆盖边表示直接的依赖关系, 而非间接的依赖关系。
- **覆盖关系(Covering Relation)**: 在 DAG 中, 覆盖关系是由所有覆盖边组成的关系。它表示了 DAG 中的直接依赖关系, 是 DAG 的一个重要子结构, 因为它简化了 DAG 的表示, 只保留了必要的边来描述顶点之间的直接关系。
- **链(Chain)**: 在 DAG 中, 一条链是一个顶点序列, 其中每个顶点都指向下一个顶点。链的长度是其包含的边的数量。

- **关键路径(Critical Path)**: 在 DAG 中, 关键路径是指最长的链, 它决定了完成所有任务所需的最短时间。
- **反链(Antichain)**: 在 DAG 中, 一个反链是一个顶点集合, 其中没有任何两个顶点之间存在有向边。反链的大小是其包含的顶点数量。
- **最大反链(Maximal Antichain)**: 在 DAG 中, 最大反链是指包含最多顶点的反链。它决定了同时可以执行的任务数量。
- **拓扑排序(Topological Sort)**: 是一种线性排序算法, 用于对 DAG 进行排序, 使得对于每条有向边(u, v), 顶点u在顶点v之前出现。拓扑排序可以通过深度优先搜索(DFS)或入度为0的顶点队列来实现。
- **狄尔沃斯引理(Dilworth's Lemma)**: 任何一个n个顶点的RAG, 对于任意数字$1 \leq k \leq n$, 都要么存在一个长度大于k的链, 要么存在一个大小至少为n/k的反链。换句话说, 在 DAG 中, 任何链(chain)的长度都小于或等于 DAG 的最长路径长度, 任何反链(antichain)的大小都小于或等于 DAG 的最大反链大小。这个引理在并行处理和任务调度中具有重要应用, 因为它提供了关于任务串行化和并行化的理论基础。
- **并行处理的极限**: 根据狄尔沃斯引理, 任务的最小串行时间由 DAG 的最长路径长度(关键路径)决定, 而完成所有任务所需的最多处理器数量由 DAG 的最大反链大小决定。这揭示了并行处理的理论极限, 即使有无限的处理器, 任务的完成时间也不能短于关键路径的长度。

偏序关系(Partial Orders):

- **严格偏序(Strict Partial Orders)**: 满足传递性和非对称性的关系, 即如果$a < b$且$b < c$, 则$a < c$; 同时如果$a < b$, 则不可能$b < a$。
- **弱偏序(Weak Partial Orders)**: 满足传递性、反对称性和自反性的关系, 即如果$a \leq b$且$b \leq c$, 则$a \leq c$; 同时如果$a \leq b$且$b \leq a$, 则$a = b$。
- **等价关系(Equivalence Relations)**: 满足自反性、对称性和传递性的关系, 即如果$a \sim a$, 则自反(Reflexive); 如果$a \sim b$, 则$b \sim a$, 则对称(Symmetric); 如果$a \sim b$且$b \sim c$, 则$a \sim c$, 则传递(Transitive)。等价关系将集合划分为互不相交的等价类, 每个等价类包含所有相互等价的元素。

2.3 通信网络 (Communication Networks)

通信网络(Communication Networks): 由交换机和线路构成的网络结构, 评估指标包括交换机数量、网络直径(延迟)和最大拥塞度。

- **网络直径(Network Diameter)**: 网络中任意两个节点之间的最长最短路径长度, 反映了网络的延迟。
- **交换数量(Switch Count)**: 网络中交换机的数量, 影响网络的成本和复杂度。
- **网络延迟(Network Latency)**: 数据包从源节点到目的节点所需的时间, 受网络直径和交换机处理时间的影响。
- **拥塞度(Congestion)**: 在网络中, 任何一条边上同时传输的数据包数量的最大值。拥塞度越高, 网络性能越差, 因为数据包可能会在交换机处排队等待传输。

完全二叉树(Complete Binary Tree): 结构简单, 但最大拥塞度极差(所有跨子网的流量都必须经过根节点)。

二维阵列(2-D Array): 通过增加交换机数量来降低拥塞度, 但代价是网络直径的增加。

蝴蝶网络(Butterfly Network): 作为树和阵列的折中方案, 利用递归结构在交换机数量、直径和拥塞度之间取得了平衡。其核心是利用FFT(快速傅里叶变换)的递归结构来设计网络, 使得数据包能够在较短的路径上进行无冲突的路由。

Beneš 网络(Beneš Network): 被誉为路由网络的“圣杯”。通过将两个蝴蝶网络背靠背连接, 实现了**最大拥塞度为 1** 的奇迹。 Beneš 网络的设计巧妙地利用了递归结构和对称性, 使得每个数据包都能找到一条无冲突的路径, 从而实现了最优的拥塞度。这种网络在理论计算机科学中具有重要意义, 因为它证明了在理想情况下, 可以设计出一个网络结构, 使得任何输入模式下的最大拥塞度都为1, 这为并行计算和通信网络的设计提供了重要的理论基础。

2.4 简单图 (Simple Graphs)

简单图(Simple Graphs): 由一组顶点和一组无向边组成的图, 用于表示事物之间的双向或对称关系。一个简单图G可以表示为$G = (V, E)$, 其中V是非空顶点集合, E是边集合, $E \subseteq \{\{u, v\} | u, v \in V, u \neq v\}$。

- **顶点的度(Degree)**: 在简单图中, 每个顶点的度是与该顶点相连的边的数量。根据图论中的**握手定理(Handshake Theorem)**, 图中所有顶点的度数之和等于边数的两倍, 即$\sum_{v \in V} \deg(v) = 2|E|$。
- **二分图(Bipartite Graphs)**: 顶点集合可以划分为两个不相交的子集U和V, 使得每条边都连接一个来自U的顶点和一个来自V的顶点, 没有奇数长度的环。
- **稳定婚姻问题(Stable Marriage)**: 给定两个大小相等的集合(如男孩和女孩), 每个成员都有一个偏好列表, 目标是找到一个匹配, 使得没有任何一对男女宁愿私奔也不愿维持现状。

- **Gale-Shapley 求偶算法(Mating Ritual)**: 通过迭代的提议和拒绝过程, 最终找到一个稳定匹配。该算法保证了男性最优的稳定匹配, 同时也是女性最差的稳定匹配。
- **匹配(Matchings)**: 在图中, 匹配是指一组边, 其中没有两条边共享一个公共顶点。匹配的大小是匹配中边的数量。一个匹配被称为完美匹配(Perfect Matching), 如果它覆盖了图中的每个顶点。
- **瓶颈引理(Bottleneck Lemma)**: 如果存在瓶颈边(即连接两个子集的边数少于子集大小), 则不存在完美匹配。
- **霍尔婚姻定理(Hall's Matching Theorem)**: 在二分图中, 存在一个完美匹配的充要条件是对于任意子集S of U, 其邻居集合$N(S)$的大小至少为$|S|$, 即$|N(S)| \geq |S|$。换句话说, 每个男孩群体喜欢的女孩总数不能少于这群男孩的数量。

同构图(Isomorphic Graphs): 两个图G和H如果存在一个双射$f : V(G) \to V(H)$, 使得对于任意两个顶点u和v, u和v在G中相邻当且仅当$f(u)$和$f(v)$在H中相邻, 则称G和H是同构的。图同构问题是一个重要的计算问题, 目前尚未找到多项式时间算法来解决它, 也没有证明它是NP完全的。

图着色(Graph Coloring): 给定一个图G, 图着色是指为图的每个顶点分配一个颜色, 使得相邻的顶点具有不同的颜色。图的色数(Chromatic Number)是指能够合法着色图G所需的最少颜色数量。图着色问题在理论计算机科学中具有重要意义, 因为它是一个NP完全问题, 这意味着没有已知的多项式时间算法能够解决所有实例的图着色问题。

连通性(Connectivity): 在简单图中, 如果存在一条路径连接任意两个顶点, 则称该图是连通的。

- **连通分量(Connected Components)**: 在一个图中, 连通分量是指一个最大连通子图, 即在该子图中任意两个顶点之间都存在路径, 但在整个图中没有其他顶点与该子图中的任何顶点相连。
- **连通图(Connected Graphs)**: 如果一个图只有一个连通分量, 则称该图是连通的。
- **K-边连通(K-Edge-Connected)**: 如果一个图在删除任意$k-1$条边后仍然保持连通, 则称该图是k-边连通的。
- **K-顶点连通(K-Vertex-Connected)**: 如果一个图在删除任意$k-1$个顶点后仍然保持连通, 则称该图是k-顶点连通的。如果一个图是k-顶点连通的, 则它也是k-边连通的。反之, 如果一个图是k-边连通的, 则它不一定是k-顶点连通的。
- **Menger's Theorem**: K-边连通的图中, 任意两个顶点之间至少存在k条边不相交的路径; K-顶点连通的图中, 任意两个顶点之间至少存在k条顶点不相交的路径。

树(Trees): 是一种连通且无环的简单图, 具有以下性质:

- 树中任意两个节点之间存在唯一路径。
- 树中边的数量总是比顶点数量少1, 即$|E| = |V| - 1$。

- 在树中, 任意添加一条边都会产生一个环。
- 树可以通过递归构造定义, 如一个树可以看作是一个根节点和若干子树的集合。

生成树(Spanning Trees): 在一个连通图G中, 生成树是一个包含G的所有顶点的子图, 同时是一个树。生成树具有以下性质:

- 生成树的边数为$|V|-1$。
- 生成树是连通的, 因为它包含了所有顶点。
- 生成树是无环的, 因为它是一个树。
- 在一个连通图中, 至少存在一棵生成树。

最小生成树(Minimum Spanning Tree): 在加权图中, 具有最小总权重的生成树。常用算法包括Prim's算法和Kruskal's算法。

- **Prim's Algorithm**: 从一个初始顶点开始, 每次选择连接已构建的生成树和未包含的顶点的最小权重边, 直到所有顶点都被包含在生成树中。
- **Kruskal's Algorithm**: 将图中的边按权重从小到大排序, 依次选择边, 只要它不形成环, 就将其加入生成树中, 直到生成树包含所有顶点。
- **Karger's Algorithm**: 一种基于随机化的算法, 用于找到图的最小割法, 从而间接找到最小生成树。。通过随机地合并边来简化图, 最终留下的两顶点之间的边数即为最小割的大小。该算法具有高概率成功找到最小割, 但需要多次运行以提高成功率。

2.5 平面图 (Planar Graphs)

平面图(Planar Graphs): 能够在二维平面上画出且边不相交的图。

欧拉公式(Euler's Formula): 平面图的核心定理, 即对于一个连通的平面图, 顶点数(v)减去边数(e)加上面数(f)等于2, 即$v-e+f=2$。欧拉公式揭示了平面图的基本结构关系, 是证明许多平面图性质的基础工具。

图的平面性限制(Planarity Constraints): 利用欧拉公式可以推导出平面图的边数上限, 进而证明某些图(如K_5(五阶完全图)和$K_{3,3}$(三村三井问题))是非平面的。

多面体分类(Polyhedra Classification): 将平面图理论应用到三维空间, 从数学上证明了世界上只能存在五种正多面体(柏拉图立体)。这些多面体分别是正四面体、正六面体(立方体)、正八面体、正十二面体和正二十面体。每种多面体都具有高度对称的结构, 并且满足欧拉公式。

四色定理(Four Color Theorem): 任何一个平面图都可以用四种颜色进行着色, 使得相邻的顶点具有不同的颜色。四色定理是图论中的一个重要结果, 它的证明依赖于计算机辅助的枚举方法, 是数学史上第一个使用计算机证明的重要定理之一。该定理在地图着色、网络设计等领域具有实际应用价值。

3 Chapter 3: Counting

计数

本书第三部分“计数”(Counting)旨在探讨计算机科学中用来分析问题规模、计算时间、存储需求以及概率论基础的核心离散数学工具。
该部分从求和与渐近分析起步, 详细讲解了各种基数(计数)规则, 最后引入了极其强大的代数工具——生成函数。

1. 求和与渐近 (Sums and Asymptotics)
2. 基数规则 (Cardinality Rules)
3. 生成函数 (Generating Functions)

3.1 求和与渐近 (Sums and Asymptotics)

求和(Sums): 在离散数学中, 求和是指对一系列数值进行加法运算的过程。求和可以表示为$\sum_{i=1}^{n} a_i$, 其中a_i是一个数列的第i项。求和在算法分析、概率论和组合数学中具有重要应用。

- **等差数列(Arithmetic Sums)**: 由一个初始值和一个公差构成的数列, 其求和公式为$\sum_{i=1}^{n}(a+(i-1)d) = \frac{n}{2}(2a+(n-1)d)$。
- **等比数列(Geometric Sums)**: 由一个初始值和一个公比构成的数列, 其求和公式为$\sum_{i=0}^{n-1} ar^i = a\frac{r^n-1}{r-1}$, 当$r \neq 1$时。
- **调和级数(Harmonic Sums)**: 定义为$H_n = \sum_{i=1}^{n} \frac{1}{i}$, 是一个重要的数列, 在算法分析中经常出现。调和级数的增长速度接近于$\ln(n)$。
- **扰动法(Perturbation Method)**: 通过将求和公式错位错项相减来获得封闭形式的一种技巧。例如, 对于等差数列的求和, 可以将$\sum_{i=1}^{n} i$与$\sum_{i=1}^{n}(i-1)$进行比较, 从而得出封闭形式。

书本叠放悖论(Book Stacking Paradox): 通过分析桌子边缘叠放书本的最大悬空距离, 得出了最大悬空量为$H_n/2$(H_n为第n个调和数), 并证明了只要书本足够多, 悬空距离可以无限大。这一悖论展示了调和级数的缓慢增长特性, 以及在实际问题中可能出现的违反直觉的结果。

积分法估算求和与乘积(Integral Method): 许多求和无法得出精确的封闭形式。可以利用微积分中的积分来作为离散求和的上下界进行极高精度的估算。

- **积分近似(Integral Approximation)**: 对于一个单调递增的函数$f(x)$, 可以通过

积分$\int_1^n f(x)dx$来估算求和$\sum_{i=1}^n f(i)$的值。具体来说, 有以下不等式成立:

$$\int_1^n f(x)dx + f(1) \leq \sum_{i=1}^n f(i) \leq \int_1^n f(x)dx + f(n)$$

- 对于调和级数H_n, 可以通过积分$\int_1^n \frac{1}{x}dx$来估算其值, 从而得出$H_n \approx \ln(n) + \gamma$ (其中γ是欧拉-马歇罗尼常数)。这一近似揭示了调和级数的增长速度与自然对数函数的关系, 并且在算法分析中经常被用来估算某些算法的时间复杂度。
- 通过取对数, 乘积可以转化为求和。利用这种方法结合积分近似, 推导出了极其重要的用于估算阶乘$n!$的**斯特林公式(Stirling's Formula)**, 即$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$。斯特林公式提供了一个非常精确的阶乘近似, 在组合数学、概率论和统计学中具有广泛应用。

渐近记号(Asymptotic Notation): 用于描述算法复杂度的符号, 包括大 O (O)、小 o (o)、Theta (Θ)、大 Omega (Ω) 等。

- **大 O 记号(Big O Notation)**: 用于描述一个函数的上界, 即存在常数C和n_0, 使得对于所有$n \geq n_0$, $f(n) \leq Cg(n)$, 则称$f(n) = O(g(n))$。
- **小 o 记号(Little o Notation)**: 用于描述一个函数的严格上界, 即对于所有常数$C > 0$, 存在n_0, 使得对于所有$n \geq n_0$, $f(n) < Cg(n)$, 则称$f(n) = o(g(n))$。
- **Theta 记号(Theta Notation)**: 用于描述一个函数的紧确界, 即存在常数C_1, C_2和n_0, 使得对于所有$n \geq n_0$, $C_1g(n) \leq f(n) \leq C_2g(n)$, 则称$f(n) = \Theta(g(n))$。
- **大 Omega 记号(Big Omega Notation)**: 用于描述一个函数的下界, 即存在常数C和n_0, 使得对于所有$n \geq n_0$, $f(n) \geq Cg(n)$, 则称$f(n) = \Omega(g(n))$。
- **小 Omega 记号(Little Omega Notation)**: 用于描述一个函数的严格下界, 即对于所有常数$C > 0$, 存在n_0, 使得对于所有$n \geq n_0$, $f(n) > Cg(n)$, 则称$f(n) = \omega(g(n))$。
- 在使用渐近记号时, 常常忽略常数因子和低阶项, 因为它们在输入规模增长时对函数的增长率影响较小。然而, 在某些情况下, 忽略这些因素可能会导致误解或错误的结论, 因此需要谨慎使用渐近记号。
- **渐近记号的逻辑陷阱**: 在使用渐近记号时, 可能会遇到一些常见的逻辑陷阱, 如误将$O(f(n))$理解为$f(n)$的确切增长率, 或者错误地比较不同类型的函数(如多项式函数和指数函数)。因此, 在分析算法复杂度时, 需要仔细考虑函数的性质和增长行为, 以避免这些陷阱。

3.2 基数规则 (Cardinality Rules)

双射法则(Bijection Rule): 如果存在一个双射(即一一对应)从集合A到集合B, 则集合A和集合B的大小相等, 即$|A| = |B|$。通过构造一个双射, 可以将一个复杂的计数问题转化为一个更简单的计数问题。

加法法则(Sum Rule): 如果一个事件可以通过选择一个来自集合A的元素或一个来自集合B的元素来完成, 且A和B没有公共元素, 则总的组合数为$|A| + |B|$。加法法则适用于不相交集合的并集, 是计数中常用的工具之一。

乘法法则(Product Rule): 如果一个事件可以通过选择一个来自集合A的元素和一个来自集合B的元素来完成, 则总的组合数为$|A| \times |B|$。乘法法则适用于多个独立选择的组合, 是计数中最基本的工具之一。

广义乘法法则(Generalized Product Rule): 当一个事件可以通过一系列选择来完成, 每个选择的可选项数量可能依赖于前面的选择, 则总的组合数为每个选择的可选项数量的乘积。例如, 计算没有重复数字的密码、棋盘放置问题和排列数等问题都可以使用广义乘法法则来解决。

除法法则(Division Rule): 当一个事件可以通过多对一的映射来完成, 即每个结果对应多个输入时, 则总的组合数为输入数量除以每个结果对应的输入数量。例如, 计算圆桌座位安排时, 由于旋转导致的重复计数属于n-to-1映射, 因此需要除以n来修正。同样地, 在德州扑克中计算“两对(Two Pairs)”牌型时, 由于两对的先后顺序在结果上没有区别, 属于2对1映射, 必须除以2来修正。

记账员法则(Bookkeeper Rule): 当一个事件可以通过排列包含不可区分元素的对象来完成时, 则总的组合数为所有对象的排列数除以每个不可区分元素的排列数。例如, 计算单词 ”BOOKKEEPER” 的字母重排时, 由于字母O、K和E分别出现了2次、2次和3次, 需要除以2!、2!和3!来修正重复计数, 从而得出总的组合数为$\frac{10!}{1!2!2!3!1!1!}$。记账员法则是一个极为实用的多项式系数法则, 在组合数学中具有广泛应用。通过记账员法则, 可以自然引出二项式定理多项式定理。

二项式定理(Binomial Theorem): 对于任何非负整数n, $(x+y)^n$可以展开为$\sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k$, 其中$\binom{n}{k}$是二项式系数, 表示从$n$个元素中选择$k$个元素的组合数。二项式定理是组合数学中的一个重要结果, 它在概率论、统计学和代数中具有广泛应用。

多项式定理(Polynomial Theorem): 对于任何非负整数n和任意数量的变量$x_1, x_2, \ldots, x_m$, $(x_1 + x_2 + \cdots + x_m)^n$可以展开为$\sum_{k_1+k_2+\cdots+k_m=n} \binom{n}{k_1, k_2, \ldots, k_m} x_1^{k_1} x_2^{k_2} \ldots x_m^{k_m}$, 其中$\binom{n}{k_1, k_2, \ldots, k_m}$是多项式系数, 表示将$n$个元素分成$m$个组, 每组包含$k_i$个元素的组合数。多项式定理是二项式定理的推广, 在组合数学和概率论中具有重要应用。

鸽巢原理(Pigeonhole Principle): 如果将n个鸽子放入m个鸽巢中, 且$n > m$, 则至少有一个鸽巢中至少有两个鸽子。鸽巢原理是一个简单但强大的工具, 用于证明存在性结果。鸽巢原理是一个非构造性的证明方法, 因为它只证明了某个对象的存在性, 而没有提供具体的构造方法。

容斥原理(Inclusion-Exclusion Principle): 用于计算多个相互重叠(非不相交)集合的并集大小。规则是交替加上和减去多重交集的大小。例如, 对于三个集合A, B和C, 容斥原理的公式为$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$。容斥原理在组合数学、概率论和统计学中具有广泛应用, 是解决复杂计数问题的重要工具。

组合证明(Combinatorial Proofs): 通过给等式两边赋予同一个现实集合的不同计数方式, 利用逻辑推演证明代数恒等式的一种方法。例如, 通过给TA分配职位证明帕斯卡三角恒等式$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$, 可以将左边的组合数解释为从$n$个学生中选择$k$个学生的方式, 而右边的组合数则可以解释为先选择一个特定的学生(如Alice), 然后根据是否选择Alice来分别计算剩余学生的组合数。通过这种方式, 可以直观地理解和证明组合恒等式。

3.3 生成函数 (Generating Functions)

生成函数(Generating Functions): 生成函数将序列$f_0, f_1, f_2, \ldots$ 作为无穷级数 $f_0 + f_1 x + f_2 x^2 + \ldots$ 中的系数。通过对级数进行加、乘、求导等代数运算, 可以直接操纵底层序列。生成函数在组合数学、概率论和算法分析中具有重要应用, 是解决复杂计数问题的强大工具之一。

卷积规则与计数(The Convolution Rule): 当把两组生成函数相乘时, 对应的系数呈现卷积形式, 这完美契合了“从两个互斥集合中挑选组合”的计数逻辑。例如, 计算有限制的苹果和香蕉(如苹果只能偶数个, 香蕉只能是5的倍数)的组合数量, 即使条件极其复杂(如荒谬的水果装袋问题), 通过生成函数相乘和代数化简也能轻易得出极简单的封闭解。

部分分式展开(Partial Fractions): 当生成函数表示为多项式商时, 利用微积分中的部分分式法将其拆解为简单的几何级数之和, 从而轻松提取出x^n的系数。部分分式展开是一个重要的技巧, 可以将复杂的生成函数分解成更简单的形式, 使得提取特定项的系数变得更加容易。

求解线性递推关系(Solving Linear Recurrences): 运用生成函数为递归算法或递归数列推导闭式解(Closed formula)。讲座中详细展示了如何通过生成函数, 将著名的“斐波那契数列(Fibonacci Numbers)”和“汉诺塔(Towers of Hanoi)”的步数计算转化为代数表达式, 并求解出精准的非递归封闭公式。通过生成函数, 可以将递推关系转化为代数方程, 从而利用代数方法求解出递推关系的闭式解。这种方法在分析算法的时间复杂度和空间复杂度时非常有用, 因为许多算法的性能可以通过递推关系来描述。